



Linear Regression

These slides were assembled by Eric Eaton, with grateful acknowledgement of the many others who made their course materials freely available online. Feel free to reuse or adapt these slides for your own academic purposes, provided that you include proper attribution. Please send comments and corrections to Eric.

Vectorization

- Consider our model for m instances:

$$h\left(\mathbf{x}^{(i)}\right)=\sum_{j=0}^n \theta_j x_j^{(i)}$$

- Let

$$\boldsymbol{\theta}=\left[\begin{array}{c} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n'} \end{array}\right] \quad \mathbf{X}=\left[\begin{array}{cccc} 1 & x_1^{(1)} & \cdots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \cdots & x_n^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \cdots & x_n^{(m)} \end{array}\right]$$

$\mathbb{R}^{(n+1) \times 1}$
 $\mathbb{R}^{m \times (n+1)}$

- Can write the model in vectorized form as $h_{\boldsymbol{\theta}}(\mathbf{x})=\mathbf{X} \boldsymbol{\theta}$

Vectorization

- For the linear regression cost function:

$$J(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^m \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2$$

$$= \frac{1}{2} \sum_{i=1}^m \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

$$= \frac{1}{2} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

$\mathbb{R}^{m \times (n'+1)}$

$\mathbb{R}^{(n'+1) \times 1}$

$\mathbb{R}^{1 \times m}$

$\mathbb{R}^{m \times 1}$

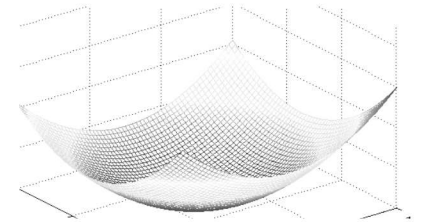
Let:

$$\mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

Closed Form Solution

- Instead of using GD, solve for optimal θ analytically

– Notice that the solution is when $\frac{\partial}{\partial \theta} J(\theta) = 0$



- Derivation:

$$\begin{aligned} \mathcal{J}(\theta) &= \frac{1}{2} (\mathbf{X}\theta - \mathbf{y})^\top (\mathbf{X}\theta - \mathbf{y}) \\ &\propto \theta^\top \mathbf{X}^\top \mathbf{X} \theta - \boxed{\mathbf{y}^\top \mathbf{X} \theta} - \boxed{\theta^\top \mathbf{X}^\top \mathbf{y}} + \mathbf{y}^\top \mathbf{y} \\ &\propto \theta^\top \mathbf{X}^\top \mathbf{X} \theta - 2\theta^\top \mathbf{X}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y} \end{aligned}$$

Note: In the original image, arrows point from the '1 x 1' scalar dimension label to the boxed terms $\mathbf{y}^\top \mathbf{X} \theta$ and $\theta^\top \mathbf{X}^\top \mathbf{y}$.

Take derivative and set equal to 0, then solve for θ :

$$\frac{\partial}{\partial \theta} (\theta^\top \mathbf{X}^\top \mathbf{X} \theta - 2\theta^\top \mathbf{X}^\top \mathbf{y} + \cancel{\mathbf{y}^\top \mathbf{y}}) = 0$$

$$(\mathbf{X}^\top \mathbf{X})\theta - \mathbf{X}^\top \mathbf{y} = 0$$

$$(\mathbf{X}^\top \mathbf{X})\theta = \mathbf{X}^\top \mathbf{y}$$

Closed Form Solution:

$$\theta = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Closed Form Solution

- Can obtain θ by simply plugging X and y into

$$\theta = (X^T X)^{-1} X^T y$$
$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(i)} & \dots & x_n^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(m)} & \dots & x_n^{(m)} \end{bmatrix} \quad y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

- If $X^T X$ is not invertible (i.e., singular), may need to:
 - Use pseudo-inverse instead of the inverse
 - In python, `numpy.linalg.pinv(a)`
 - Remove redundant (not linearly independent) features
 - Remove extra features to ensure that $n \leq m$

Gradient Descent vs Closed Form

Gradient Descent

- Requires multiple iterations
- Need to choose α
- Works well when m is large
- Can support incremental learning

Closed Form Solution

- Non-iterative
- No need for α
- Slow if m is large
 - Computing $(X^T X)^{-1}$ is roughly $O(m^3)$